cms_doubleelectron_ppee_QED0

June 3, 2026

1 OTUS | CMS DoubleElectron with MG5 Z/γ^* Prior

This notebook trains OTUS using CMS Open Data DoubleElectron events as detector-level data `x`, and a standalone MadGraph5 pre-detector Drell–Yan `pp -> e+ e-` prior as latent/theory-level data `z`.

Expected `z` HDF5 file layout:

```
FDL/zData : shape (N, 8)
columns   : [e- px, e- py, e- pz, e- E, e+ px, e+ py, e+ pz, e+ E]
```

Put the MG5 HDF5 file in the same folder as this notebook, or set `theory_prior_file` to its full path in the meta-parameter cell.

2 Load Required Libraries

[76]:
```
%matplotlib inline
from pathlib import Path

import awkward as ak
import matplotlib.pyplot as plt
import numpy as np
import torch
import uproot

import sys
from pathlib import Path

import os

os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"

# Change this to your actual OTUS root folder
OTUS_ROOT = Path(r"C:\Users\AhrixMarin\Desktop\otus")

UTILITY_DIR = OTUS_ROOT / "utilityFunctions"
```

```
assert (UTILITY_DIR / "func_utils.py").exists(), f"Cannot find func_utils.py in␣
 ↪{UTILITY_DIR}"

sys.path.insert(0, str(UTILITY_DIR))

print("Using utilityFunctions from:", UTILITY_DIR)

#-- Determine if using GPU or CPU --#
import os
os.environ['CUDA_VISIBLE_DEVICES'] = '0'  # Set to '-1' to disable GPU

from configs import device

print('Using device:', device)
```

```
Using utilityFunctions from: C:\Users\AhrixMarin\Desktop\otus\utilityFunctions
Using device: cuda
```

3 Meta Parameters

[77]:
```
cms_root_file = Path('../cms_zpeak/data/Run2012B_DoubleElectron.root')

# MG5 prior produced on Linux and copied to this notebook folder.
# Change this filename if your HDF5 file has a different name.
theory_prior_filename = 'cms_dyee_mg5_8tev_fiducial_70_110.hdf5'
theory_prior_file = Path(theory_prior_filename)

# Used only for saved model/results filenames.
dataset_name = 'cms_doubleelectron_mg5_dyee'

#-- Set random seeds --#
seed = 0
torch.manual_seed(seed)
np.random.seed(seed)

#-- Set data type --#
from configs import float_type
print('Using data type:', float_type)
print('Notebook/kernel working directory:', Path.cwd())
print('Expected MG5 prior file:', theory_prior_file)
print('Local .hdf5 files:', sorted([p.name for p in Path('.').glob('*.hdf5')]))
```

```
Using data type: float32
Notebook/kernel working directory:
c:\Users\AhrixMarin\Desktop\otus\experiments\cms_doubleelectron
Expected MG5 prior file: cms_dyee_mg5_8tev_fiducial_70_110.hdf5
Local .hdf5 files: ['cms_dyee_mg5_8tev_dy1j_ptj5_fiducial_70_110.hdf5',
```

```
'cms_dyee_mg5_8tev_fiducial_70_110.hdf5']
```

4 Load CMS DoubleElectron ROOT File

[78]:
```
assert cms_root_file.exists(), f'File not found: {cms_root_file.resolve()}'

cms_file = uproot.open(cms_root_file)
events = cms_file['Events']

print('File:', cms_root_file.resolve())
print('Number of events:', events.num_entries)

electron_branches = [k for k in events.keys() if 'Electron' in k]
electron_branches[:30]
```

```
File: C:\Users\AhrixMarin\Desktop\otus\experiments\cms_zpeak\data\Run2012B_Doubl
eElectron.root
Number of events: 21474287
```

[78]:
```
['nElectron',
 'Electron_pt',
 'Electron_eta',
 'Electron_phi',
 'Electron_mass',
 'Electron_charge',
 'Electron_pfRelIso03_all',
 'Electron_dxy',
 'Electron_dxyErr',
 'Electron_dz',
 'Electron_dzErr']
```

[79]:
```
branches = [
    'nElectron',
    'Electron_pt',
    'Electron_eta',
    'Electron_phi',
    'Electron_mass',
    'Electron_charge',
    'Electron_pfRelIso03_all',
    'Electron_dxy',
    'Electron_dz',
]

arrays = events.arrays(branches, library='ak')

print('Loaded events:', len(arrays['nElectron']))
print('Example nElectron values:', arrays['nElectron'][:10])
```

```
Loaded events: 21474287
Example nElectron values: [0, 1, 2, 2, 1, 2, 0, 2, 1, 0]
```

5 Select Good Electron Pairs

[80]:
```
# Basic kinematic cuts
electron_pt_min = 20.0
electron_abs_eta_max = 2.5

# Electron quality cuts
electron_iso_max = 0.15
electron_dxy_max = 0.05
electron_dz_max = 0.10

# Z-window cut
z_mass_min = 70.0
z_mass_max = 110.0

electrons = ak.zip({
    'pt': arrays['Electron_pt'],
    'eta': arrays['Electron_eta'],
    'phi': arrays['Electron_phi'],
    'mass': arrays['Electron_mass'],
    'charge': arrays['Electron_charge'],
    'pfRelIso03_all': arrays['Electron_pfRelIso03_all'],
    'dxy': arrays['Electron_dxy'],
    'dz': arrays['Electron_dz'],
})

abs_eta = np.abs(electrons.eta)
outside_ecal_gap = ~((abs_eta > 1.4442) & (abs_eta < 1.566))

selected_electrons = electrons[
    (electrons.pt > electron_pt_min)
    & (abs_eta < electron_abs_eta_max)
    & outside_ecal_gap
    & (electrons.pfRelIso03_all < electron_iso_max)
    & (np.abs(electrons.dxy) < electron_dxy_max)
    & (np.abs(electrons.dz) < electron_dz_max)
]

print('Total selected electrons:', ak.sum(ak.num(selected_electrons)))
print('Events with >=2 selected electrons:', ak.sum(ak.num(selected_electrons)␣
 ↪>= 2))

pairs = ak.combinations(selected_electrons, 2, fields=['e1', 'e2'])
os_pairs = pairs[(pairs.e1.charge * pairs.e2.charge) < 0]
```

```
e_minus = ak.where(os_pairs.e1.charge < 0, os_pairs.e1, os_pairs.e2)
e_plus = ak.where(os_pairs.e1.charge > 0, os_pairs.e1, os_pairs.e2)

def pair_mass(first, second):
    px = first.pt * np.cos(first.phi) + second.pt * np.cos(second.phi)
    py = first.pt * np.sin(first.phi) + second.pt * np.sin(second.phi)
    pz = first.pt * np.sinh(first.eta) + second.pt * np.sinh(second.eta)
    e1 = np.sqrt((first.pt * np.cosh(first.eta)) ** 2 + first.mass ** 2)
    e2 = np.sqrt((second.pt * np.cosh(second.eta)) ** 2 + second.mass ** 2)
    mass2 = (e1 + e2) ** 2 - px ** 2 - py ** 2 - pz ** 2
    return np.sqrt(ak.where(mass2 > 0, mass2, 0))

m_ee = pair_mass(e_minus, e_plus)
z_window = (m_ee > z_mass_min) & (m_ee < z_mass_max)

e_minus = ak.flatten(e_minus[z_window])
e_plus = ak.flatten(e_plus[z_window])
masses_ee = ak.to_numpy(ak.flatten(m_ee[z_window]))

print('Opposite-sign dielectron pairs in 70-110 GeV:', len(masses_ee))
print(f'Mean mass:   {np.mean(masses_ee):.6f} GeV')
print(f'Median mass: {np.median(masses_ee):.6f} GeV')
```

```
Total selected electrons: 9734445
Events with >=2 selected electrons: 1903393
Opposite-sign dielectron pairs in 70-110 GeV: 1344681
Mean mass:   90.638089 GeV
Median mass: 91.148434 GeV
```

6 Build Detector-Level x-Space Array

[81]:
```
def p4_array(particle):
    pt = ak.to_numpy(particle.pt)
    eta = ak.to_numpy(particle.eta)
    phi = ak.to_numpy(particle.phi)
    mass = ak.to_numpy(particle.mass)

    px = pt * np.cos(phi)
    py = pt * np.sin(phi)
    pz = pt * np.sinh(eta)
    energy = np.sqrt(px ** 2 + py ** 2 + pz ** 2 + mass ** 2)
    return np.stack([px, py, pz, energy], axis=1)

# Match ppzee ordering: [p_e-^mu, p_e+^mu], with p^mu = [px, py, pz, E]
x_data = np.concatenate([p4_array(e_minus), p4_array(e_plus)], axis=1)
```

```
print('x_data shape:', x_data.shape)
print('First x row:', x_data[0])
```

```
x_data shape: (1344681, 8)
First x row: [-35.98058   48.777832  35.590557  70.28917   20.748224 -27.40425
  26.977688  43.69528 ]
```

7 Load Theory-Level z-Space Prior

[82]:
```
import h5py

# Load the MadGraph5 pre-detector physics prior z from the HDF5 file.
# Expected HDF5 layout:
#   FDL/zData : shape (N, 8)
# with columns:
#   [e- px, e- py, e- pz, e- E, e+ px, e+ py, e+ pz, e+ E]

if not theory_prior_file.exists():
    available_hdf5 = sorted([p.name for p in Path('.').glob('*.hdf5')])
    print('Available .hdf5 files in current working directory:', available_hdf5)
    raise FileNotFoundError(
        f'Cannot find {theory_prior_file}. '
        'Either move the HDF5 file into the notebook folder, '
        'or set theory_prior_file to the absolute Windows path.'
    )

with h5py.File(theory_prior_file, 'r') as f:
    print('HDF5 top-level keys:', list(f.keys()))
    if 'FDL' in f and isinstance(f['FDL'], h5py.Group) and 'zData' in f['FDL']:
        z_data = np.asarray(f['FDL/zData'])
    elif 'zData' in f:
        z_data = np.asarray(f['zData'])
    else:
        raise KeyError('Could not find z prior. Expected FDL/zData or zData in␣
 ↪the HDF5 file.')

# Keep the same 8D ordering used by x_data:
# [p_e-^mu, p_e+^mu], with p^mu = [px, py, pz, E]
z_data = z_data[:, :8]

assert x_data.ndim == 2 and x_data.shape[1] == 8, f'x_data should have shape␣
 ↪(N, 8), got {x_data.shape}'
assert z_data.ndim == 2 and z_data.shape[1] == 8, f'z_data should have shape␣
 ↪(N, 8), got {z_data.shape}'
assert np.all(np.isfinite(x_data)), 'x_data contains NaN or inf'
assert np.all(np.isfinite(z_data)), 'z_data contains NaN or inf'
```

```
# OTUS training uses unpaired samples from p(x) and p(z). Use the common sample count.
num_samples = min(len(x_data), len(z_data))
x_data = x_data[:num_samples]
z_data = z_data[:num_samples]

x_dim = int(x_data.shape[1])
z_dim = int(z_data.shape[1])

print('Loaded z prior from:', theory_prior_file)
print('z_data shape:', z_data.shape)
print('x_data shape:', x_data.shape)
print('num_samples used:', num_samples)
print('First z row:', z_data[0])
```

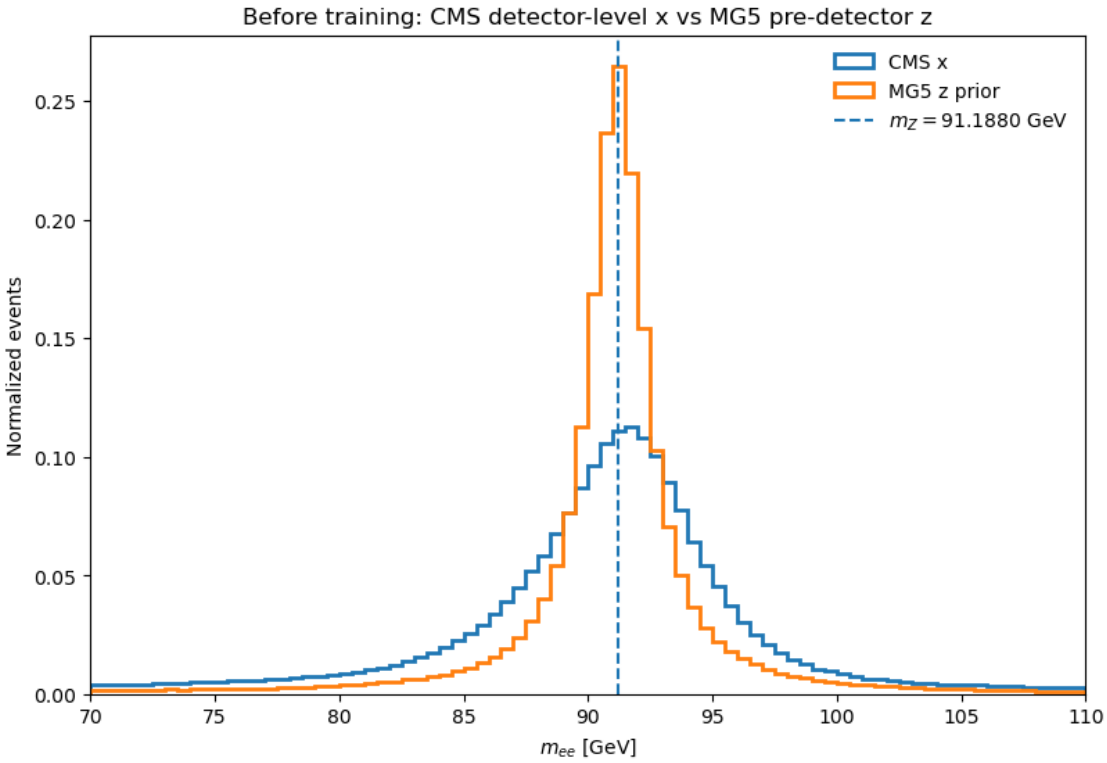
```
HDF5 top-level keys: ['FDL']
Loaded z prior from: cms_dyee_mg5_8tev_fiducial_70_110.hdf5
z_data shape: (640449, 8)
x_data shape: (640449, 8)
num_samples used: 640449
First z row: [-37.5503    12.661246  24.690422  46.689926  37.5503   -12.661246
  98.864876 106.51102 ]
```

7.1 Pre-training sanity check

Compare the CMS detector-level mass distribution with the MG5 pre-detector prior before training. The MG5 curve should usually be narrower because it does not include detector resolution, electron energy smearing, or reconstruction effects.

[83]:
```
from func_utils import Zboson_mass

x_m_before = Zboson_mass(x_data)
z_m_before = Zboson_mass(z_data)

fig, ax = plt.subplots(figsize=(8, 5.5), constrained_layout=True)
bins = np.linspace(70, 110, 81)

ax.hist(x_m_before, bins=bins, histtype='step', linewidth=2, density=True, label='CMS x')
ax.hist(z_m_before, bins=bins, histtype='step', linewidth=2, density=True, label='MG5 z prior')
ax.axvline(91.1880, linestyle='--', linewidth=1.5, label=r'$m_Z = 91.1880$ GeV')

ax.set_xlabel(r'$m_{ee}$ [GeV]')
ax.set_ylabel('Normalized events')
ax.set_xlim(70, 110)
ax.set_title('Before training: CMS detector-level x vs MG5 pre-detector z')
```

```
ax.legend(frameon=False)
plt.show()

print(f'CMS x mass mean: {np.mean(x_m_before):.4f} GeV')
print(f'MG5 z mass mean: {np.mean(z_m_before):.4f} GeV')
print(f'CMS x mass std:  {np.std(x_m_before):.4f} GeV')
print(f'MG5 z mass std:  {np.std(z_m_before):.4f} GeV')
```




```
CMS x mass mean: 90.6363 GeV
MG5 z mass mean: 91.0066 GeV
CMS x mass std:  5.8516 GeV
MG5 z mass std:  4.0704 GeV
```

7.1.1 Define target invariant masses

[84]:
```
# Electrons are treated as massless for consistency with ppzee.ipynb.
x_inv_masses = np.zeros(2)
z_inv_masses = np.zeros(2)
```

8 Train/Test Split

[85]:
```
from torch.utils.data import DataLoader

rng = np.random.default_rng(seed)
x_perm = rng.permutation(len(x_data))
z_perm = rng.permutation(len(z_data))
x_data = x_data[x_perm]
z_data = z_data[z_perm]

train_ratio = 0.80
val_ratio = 0.10

train_size = int(num_samples * train_ratio)
val_size = int(num_samples * val_ratio)
test_size = num_samples - train_size - val_size

x_train = x_data[:train_size]
x_val = x_data[train_size:train_size + val_size]
x_test = x_data[train_size + val_size:]

z_train = z_data[:train_size]
z_val = z_data[train_size:train_size + val_size]
z_test = z_data[train_size + val_size:]

x_train, x_val, x_test, z_train, z_val, z_test = list(map(
    lambda arr: arr.astype(float_type),
    [x_train, x_val, x_test, z_train, z_val, z_test],
))

x_train_mean, x_train_std = np.mean(x_train, axis=0), np.std(x_train, axis=0)
z_train_mean, z_train_std = np.mean(z_train, axis=0), np.std(z_train, axis=0)

eval_batch_size = min(20000, max(1, len(x_val)))
eval_loaders = DataLoader(dataset=x_val, batch_size=eval_batch_size,␣
 ↪shuffle=True), \
              DataLoader(dataset=z_val, batch_size=eval_batch_size,␣
 ↪shuffle=True)

print('z_train shape, x_train shape:', z_train.shape, x_train.shape)
print('z_val   shape, x_val   shape:', z_val.shape, x_val.shape)
print('z_test  shape, x_test  shape:', z_test.shape, x_test.shape)
```

```
z_train shape, x_train shape: (512359, 8) (512359, 8)
z_val   shape, x_val   shape: (64044, 8) (64044, 8)
z_test  shape, x_test  shape: (64046, 8) (64046, 8)
```

9 Train

9.1 Import Training Specific Libraries and Functions

[86]:
```
from torch import optim
from ppzee_utils import train_and_val
```

9.2 Central hyperparameter block

Edit this one cell when tuning architecture, batch sizes, and staged training.

[87]:
```
# ============================================================
# Central Hyperparameter Block
# Edit this cell when tuning the training.
# ============================================================

HP = {
    # -------------------------
    # Model architecture
    # -------------------------
    "cond_noise": True,
    "num_hidden_layers": 1,
    "dim_per_hidden_layer": 128,
    "activation": torch.nn.ReLU,
    "sigma_fun": "softplus",

    # -------------------------
    # DataLoader settings
    # Smaller batches give more optimizer updates per epoch.
    # -------------------------
    "train_batch_size": 4096,
    "eval_batch_size": 20000,

    # -------------------------
    # Training stages
    # beta: x reconstruction loss for x -> z -> x
    # lamb: latent-prior matching loss between encode(x) and z
    # tau: direct generator loss between decode(z) and CMS x
    # nu_e / nu_d: anchor losses for encoder/decoder
    # -------------------------
    "stages": [
        {
            "name": "stage1_anchor_warmup",
            "enabled": True,
            "epochs": 50,
            "lr": 1e-3,
            "beta": 1.0,
            "lamb": 1.0,
```

```
            "tau": 1.0,
            "rho": 0.0,
            "nu_e": 50.0,
            "nu_d": 50.0,
            "num_slices": 1000,
            "log_freq": 10,
            "lr_decay": False,
        },
        {
            "name": "stage2_joint_no_anchor",
            "enabled": True,
            "epochs": 100,
            "lr": 1e-3,
            "beta": 1.0,
            "lamb": 1.0,
            "tau": 1.0,
            "rho": 0.0,
            "nu_e": 0.0,
            "nu_d": 0.0,
            "num_slices": 1000,
            "log_freq": 20,
            "lr_decay": True,
        },
        {
            "name": "stage3_direct_z_to_x",
            "enabled": True,
            "epochs": 200,
            "lr": 1e-4,
            "beta": 0.5,
            "lamb": 0.5,
            "tau": 3.0,
            "rho": 0.0,
            "nu_e": 0.0,
            "nu_d": 0.0,
            "num_slices": 500,
            "log_freq": 20,
            "lr_decay": True,
        },
    ],
}

if HP["cond_noise"]:
    from models import CondNoiseAutoencoder
    Autoencoder = CondNoiseAutoencoder
else:
    from models import Autoencoder
```

```
print('Using model class:', Autoencoder.__name__)
print('Enabled stages:', [s['name'] for s in HP['stages'] if s.get('enabled',␣
  ↪True)])
```

```
Using model class: CondNoiseAutoencoder
Enabled stages: ['stage1_anchor_warmup', 'stage2_joint_no_anchor',
'stage3_direct_z_to_x']
```

9.3 Build model from hyperparameter block

[88]:
```
# ============================================================
# Build Model from HP
# ============================================================

num_hidden_layers = HP["num_hidden_layers"]
dim_per_hidden_layer = HP["dim_per_hidden_layer"]
hidden_layer_dims = num_hidden_layers * [dim_per_hidden_layer]

activation = HP["activation"]
sigma_fun = HP["sigma_fun"]

model = Autoencoder(
    x_dim=x_dim,
    z_dim=z_dim,
    hidden_layer_dims=hidden_layer_dims,
    raw_io=True,
    x_stats=np.stack([x_train_mean, x_train_std]),
    z_stats=np.stack([z_train_mean, z_train_std]),
    x_inv_masses=x_inv_masses,
    z_inv_masses=z_inv_masses,
    stoch_enc=True,
    stoch_dec=True,
    activation=activation,
    sigma_fun=sigma_fun,
)

model
```

[88]:
```
CondNoiseAutoencoder(
  (encoder): CondNoiseMLP(
    (sigma_fun): Softplus(beta=1.0, threshold=20.0)
    (output_nn): Sequential(
      (0): Linear(in_features=14, out_features=128, bias=True)
      (1): ReLU()
      (2): Linear(in_features=128, out_features=6, bias=True)
    )
    (cond_noise_nn): Sequential(
```

```
      (0): Linear(in_features=8, out_features=128, bias=True)
      (1): ReLU()
      (2): Linear(in_features=128, out_features=12, bias=True)
    )
  )
  (decoder): CondNoiseMLP(
    (sigma_fun): Softplus(beta=1.0, threshold=20.0)
    (output_nn): Sequential(
      (0): Linear(in_features=14, out_features=128, bias=True)
      (1): ReLU()
      (2): Linear(in_features=128, out_features=6, bias=True)
    )
    (cond_noise_nn): Sequential(
      (0): Linear(in_features=8, out_features=128, bias=True)
      (1): ReLU()
      (2): Linear(in_features=128, out_features=12, bias=True)
    )
  )
)
```

9.4 Train OTUS

[89]:
```
# ============================================================
# DataLoaders from HP
# ============================================================

train_batch_size = min(HP["train_batch_size"], max(1, len(x_train)))
eval_batch_size = min(HP["eval_batch_size"], max(1, len(x_val)))

train_loaders = DataLoader(
    dataset=x_train,
    batch_size=train_batch_size,
    shuffle=True,
), DataLoader(
    dataset=z_train,
    batch_size=train_batch_size,
    shuffle=True,
)

eval_loaders = DataLoader(
    dataset=x_val,
    batch_size=eval_batch_size,
    shuffle=True,
), DataLoader(
    dataset=z_val,
    batch_size=eval_batch_size,
    shuffle=True,
```

```
)

print('train_batch_size:', train_batch_size)
print('eval_batch_size:', eval_batch_size)
print('training batches per epoch:', min(len(train_loaders[0]),␣
 ↪len(train_loaders[1])))
```

```
train_batch_size: 4096
eval_batch_size: 20000
training batches per epoch: 126
```

[90]:
```
%%time

# ============================================================
# Train OTUS using stages from HP
# ============================================================

history = None
eval_losses = None

for stage in HP["stages"]:
    if not stage.get("enabled", True):
        print(f"Skipping disabled stage: {stage['name']}")
        continue

    print("\n" + "=" * 80)
    print("Starting training stage:", stage["name"])
    print("=" * 80)

    config = {
        "num_hidden_layers": HP["num_hidden_layers"],
        "dim_per_hidden_layer": HP["dim_per_hidden_layer"],
        "epochs": stage["epochs"],
        "lr": stage["lr"],
        "beta": stage["beta"],
        "lamb": stage["lamb"],
        "tau": stage["tau"],
        "rho": stage["rho"],
        "nu_e": stage["nu_e"],
        "nu_d": stage["nu_d"],
        "num_slices": stage["num_slices"],
    }

    optimizer = optim.Adam(model.parameters(), lr=config["lr"])

    eval_losses, history = train_and_val(
        model,
```

```
        train_loaders,
        eval_loaders,
        config,
        optimizer,
        verbose=True,
        prev_hist=history,
        log_freq=stage["log_freq"],
        lr_decay=stage["lr_decay"],
    )

print("\nTraining complete.")
print("Final eval losses:", eval_losses)
```

```
================================================================================
Starting training stage: stage1_anchor_warmup
================================================================================
{'num_hidden_layers': 1, 'dim_per_hidden_layer': 128, 'epochs': 50, 'lr': 0.001,
'beta': 1.0, 'lamb': 1.0, 'tau': 1.0, 'rho': 0.0, 'nu_e': 50.0, 'nu_d': 50.0,
'num_slices': 1000}
epoch:  0
train -- loss:784.318, x_loss:374.875, z_loss:94.6232, alt_x_loss:228.09,
x_constraint_loss:0, anchor_loss:1.7346
eval -- loss:282.631, x_loss:390.924, z_loss:81.3081, alt_x_loss:201.322,
anchor_loss:1.7921
epoch:  10
train -- loss:173.294, x_loss:8.07926, z_loss:51.309, alt_x_loss:97.2614,
x_constraint_loss:0, anchor_loss:0.332891
eval -- loss:53.9188, x_loss:11.6338, z_loss:14.8158, alt_x_loss:39.1029,
anchor_loss:0.324912
epoch:  20
train -- loss:216.454, x_loss:5.27676, z_loss:80.2616, alt_x_loss:117.991,
x_constraint_loss:0, anchor_loss:0.258501
eval -- loss:49.4095, x_loss:6.46558, z_loss:13.1104, alt_x_loss:36.2992,
anchor_loss:0.238319
epoch:  30
train -- loss:146.696, x_loss:5.89462, z_loss:41.0802, alt_x_loss:91.0748,
x_constraint_loss:0, anchor_loss:0.172933
eval -- loss:42.3857, x_loss:7.34483, z_loss:12.0068, alt_x_loss:30.3789,
anchor_loss:0.188696
epoch:  40
train -- loss:140.161, x_loss:4.98489, z_loss:34.909, alt_x_loss:94.0733,
x_constraint_loss:0, anchor_loss:0.123876
eval -- loss:33.3503, x_loss:8.2944, z_loss:9.36882, alt_x_loss:23.9814,
anchor_loss:0.148423
epoch:  49
train -- loss:198.13, x_loss:4.62527, z_loss:62.9695, alt_x_loss:125.465,
x_constraint_loss:0, anchor_loss:0.101388
```

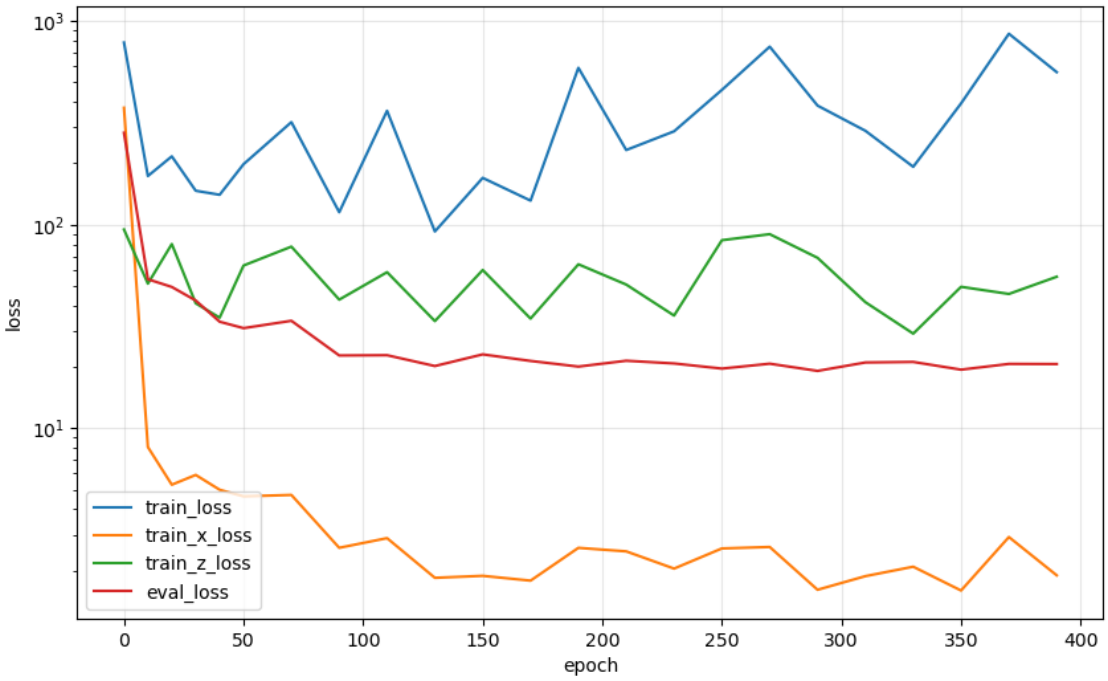
```
eval -- loss:31.0084, x_loss:6.02738, z_loss:7.60741, alt_x_loss:23.401,
anchor_loss:0.118817

================================================================================
Starting training stage: stage2_joint_no_anchor
================================================================================
{'num_hidden_layers': 1, 'dim_per_hidden_layer': 128, 'epochs': 100, 'lr':
0.001, 'beta': 1.0, 'lamb': 1.0, 'tau': 1.0, 'rho': 0.0, 'nu_e': 0.0, 'nu_d':
0.0, 'num_slices': 1000}
epoch:  0
train -- loss:318.74, x_loss:4.7012, z_loss:77.9513, alt_x_loss:236.088,
x_constraint_loss:0, anchor_loss:0
eval -- loss:33.7024, x_loss:6.38935, z_loss:7.5274, alt_x_loss:26.175,
anchor_loss:0.186799
epoch:  20
train -- loss:115.095, x_loss:2.58509, z_loss:42.805, alt_x_loss:69.7049,
x_constraint_loss:0, anchor_loss:0
eval -- loss:22.7645, x_loss:3.4349, z_loss:3.76575, alt_x_loss:18.9988,
anchor_loss:0.333598
epoch:  40
train -- loss:362.256, x_loss:2.88363, z_loss:58.3918, alt_x_loss:300.981,
x_constraint_loss:0, anchor_loss:0
eval -- loss:22.8327, x_loss:3.09262, z_loss:4.26545, alt_x_loss:18.5672,
anchor_loss:0.37403
epoch:  60
train -- loss:92.5615, x_loss:1.84109, z_loss:33.5818, alt_x_loss:57.1386,
x_constraint_loss:0, anchor_loss:0
eval -- loss:20.1997, x_loss:2.80869, z_loss:3.27219, alt_x_loss:16.9276,
anchor_loss:0.391924
epoch:  80
train -- loss:169.649, x_loss:1.88272, z_loss:59.852, alt_x_loss:107.914,
x_constraint_loss:0, anchor_loss:0
eval -- loss:23.0488, x_loss:2.63181, z_loss:3.97615, alt_x_loss:19.0727,
anchor_loss:0.403799
epoch:  99
train -- loss:131.191, x_loss:1.78321, z_loss:34.5436, alt_x_loss:94.8637,
x_constraint_loss:0, anchor_loss:0
eval -- loss:21.4009, x_loss:2.56547, z_loss:3.50341, alt_x_loss:17.8975,
anchor_loss:0.412864

================================================================================
Starting training stage: stage3_direct_z_to_x
================================================================================
{'num_hidden_layers': 1, 'dim_per_hidden_layer': 128, 'epochs': 200, 'lr':
0.0001, 'beta': 0.5, 'lamb': 0.5, 'tau': 3.0, 'rho': 0.0, 'nu_e': 0.0, 'nu_d':
0.0, 'num_slices': 500}
epoch:  0
train -- loss:589.153, x_loss:2.58388, z_loss:63.8616, alt_x_loss:185.31,
```

```
x_constraint_loss:0, anchor_loss:0
eval -- loss:20.0747, x_loss:2.58548, z_loss:2.79755, alt_x_loss:17.2771,
anchor_loss:0.413206
epoch:  20
train -- loss:232.519, x_loss:2.48287, z_loss:50.763, alt_x_loss:68.6319,
x_constraint_loss:0, anchor_loss:0
eval -- loss:21.4467, x_loss:2.65641, z_loss:3.38497, alt_x_loss:18.0617,
anchor_loss:0.422761
epoch:  40
train -- loss:287.265, x_loss:2.04308, z_loss:35.7552, alt_x_loss:89.4552,
x_constraint_loss:0, anchor_loss:0
eval -- loss:20.7996, x_loss:2.6586, z_loss:2.97712, alt_x_loss:17.8225,
anchor_loss:0.425473
epoch:  60
train -- loss:458.995, x_loss:2.56666, z_loss:83.8232, alt_x_loss:138.6,
x_constraint_loss:0, anchor_loss:0
eval -- loss:19.6241, x_loss:2.67787, z_loss:2.94412, alt_x_loss:16.68,
anchor_loss:0.425807
epoch:  80
train -- loss:748.393, x_loss:2.60918, z_loss:89.7633, alt_x_loss:234.069,
x_constraint_loss:0, anchor_loss:0
eval -- loss:20.7431, x_loss:2.68775, z_loss:3.04359, alt_x_loss:17.6995,
anchor_loss:0.428468
epoch:  100
train -- loss:384.265, x_loss:1.60831, z_loss:68.6847, alt_x_loss:116.373,
x_constraint_loss:0, anchor_loss:0
eval -- loss:19.1151, x_loss:2.67609, z_loss:2.81271, alt_x_loss:16.3024,
anchor_loss:0.428194
epoch:  120
train -- loss:288.995, x_loss:1.87598, z_loss:41.5944, alt_x_loss:89.0866,
x_constraint_loss:0, anchor_loss:0
eval -- loss:21.0329, x_loss:2.68875, z_loss:3.21215, alt_x_loss:17.8207,
anchor_loss:0.428962
epoch:  140
train -- loss:192.351, x_loss:2.0874, z_loss:29.1393, alt_x_loss:58.9126,
x_constraint_loss:0, anchor_loss:0
eval -- loss:21.1608, x_loss:2.68784, z_loss:3.83556, alt_x_loss:17.3253,
anchor_loss:0.429617
epoch:  160
train -- loss:393.935, x_loss:1.59448, z_loss:49.4956, alt_x_loss:122.797,
x_constraint_loss:0, anchor_loss:0
eval -- loss:19.3891, x_loss:2.69558, z_loss:3.16438, alt_x_loss:16.2247,
anchor_loss:0.430118
epoch:  180
train -- loss:866.419, x_loss:2.9199, z_loss:45.6541, alt_x_loss:280.711,
x_constraint_loss:0, anchor_loss:0
eval -- loss:20.6932, x_loss:2.7002, z_loss:3.4286, alt_x_loss:17.2646,
anchor_loss:0.430197
```

```
epoch:  199
train -- loss:561.12, x_loss:1.89336, z_loss:55.4602, alt_x_loss:177.481,
x_constraint_loss:0, anchor_loss:0
eval -- loss:20.6599, x_loss:2.70664, z_loss:3.38118, alt_x_loss:17.2787,
anchor_loss:0.430512

Training complete.
Final eval losses: {'x_loss': 2.7066397666931152, 'z_loss': 3.3811798095703125,
'alt_x_loss': 17.278705596923828, 'encoder_anchor_loss': 0.21493352949619293,
'decoder_anchor_loss': 0.21557815372943878, 'loss': 20.65988540649414}
CPU times: total: 1h 8min 11s
Wall time: 39min 12s
```

9.5 Plot Loss History

[91]:
```python
def history_to_numpy(values):
    """Convert history lists containing Python numbers, NumPy values, or Torch␣
 ↪tensors to NumPy."""
    out = []
    for v in values:
        if isinstance(v, torch.Tensor):
            out.append(v.detach().cpu().item() if v.ndim == 0 else v.detach().
 ↪cpu().numpy())
        else:
            out.append(v)
    return np.asarray(out, dtype=float)

epoch = history_to_numpy(history['epoch'])
train_loss = history_to_numpy(history['train_loss'])
train_x_loss = history_to_numpy(history['train_x_loss'])
train_z_loss = history_to_numpy(history['train_z_loss'])
eval_loss = history_to_numpy(history['eval_loss'])

fig = plt.figure(figsize=(10, 6))

plt.plot(epoch, train_loss, label='train_loss')
plt.plot(epoch, train_x_loss, label='train_x_loss')
plt.plot(epoch, train_z_loss, label='train_z_loss')
plt.plot(epoch, eval_loss, label='eval_loss')

plt.xlabel('epoch')
plt.ylabel('loss')
plt.yscale('log')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



10 Show Results

10.1 Get Results

[92]:
```
#-- Reset random seeds --#
torch.manual_seed(seed)
np.random.seed(seed)

#-- Convert model to CPU and evaluation mode --#
model.to('cpu')
model.encoder.output_stats.to('cpu')
model.decoder.output_stats.to('cpu')
model.eval()

all_arrs = {'train': {}, 'val': {}, 'test': {}}
all_arrs['train']['x'] = x_train
all_arrs['train']['z'] = z_train
all_arrs['val']['x'] = x_val
all_arrs['val']['z'] = z_val
all_arrs['test']['x'] = x_test
all_arrs['test']['z'] = z_test

with torch.no_grad():
    for data_key in 'train', 'val', 'test':
```

```
        arrs = all_arrs[data_key]

        z_tensor = torch.from_numpy(arrs['z'])
        x_tensor = torch.from_numpy(arrs['x'])

        arrs['z_decoded'] = model.decode(z_tensor)
        arrs['x_encoded'] = model.encode(x_tensor)
        arrs['x_reconstructed'] = model.decode(arrs['x_encoded'])

        for field, arr in list(arrs.items()):
            if isinstance(arr, torch.Tensor):
                arrs[field] = arr.detach().cpu().numpy()

print('Generated arrays for:', list(all_arrs.keys()))
for key in ['x', 'z', 'x_reconstructed', 'z_decoded']:
    print(key, all_arrs['val'][key].shape)
```

```
Generated arrays for: ['train', 'val', 'test']
x (64044, 8)
z (64044, 8)
x_reconstructed (64044, 8)
z_decoded (64044, 8)
```

11 Save Results

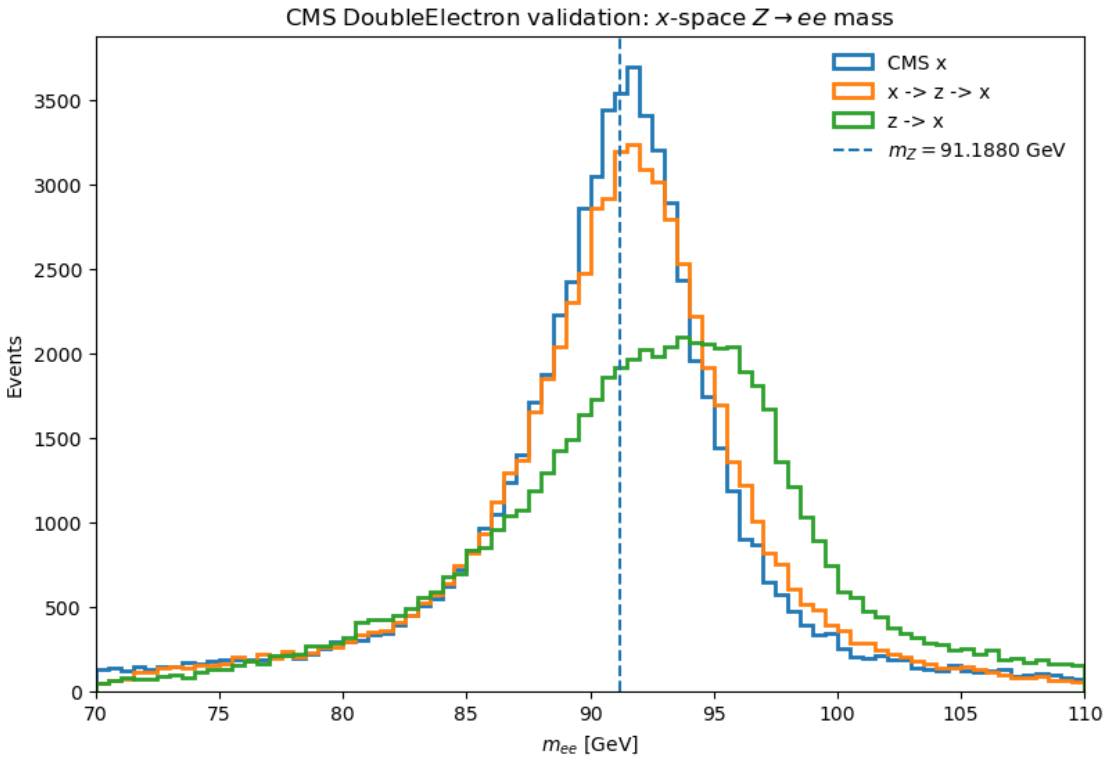
[93]:
```
model_save_path = 'otus-dataset=%s.pkl' % dataset_name
torch.save(model.state_dict(), model_save_path)
print('Saved model weights to', model_save_path)

results_save_path = 'otus_results-dataset=%s_test.npz' % dataset_name
np.savez(results_save_path, **all_arrs['test'])
print('Model results saved at', results_save_path)

history_save_path = 'otus_history-dataset=%s.npz' % dataset_name
history_np = {key: history_to_numpy(value) for key, value in history.items()}
np.savez(history_save_path, **history_np)
print('Training history saved at', history_save_path)
```

```
Saved model weights to otus-dataset=cms_doubleelectron_mg5_dyee.pkl
Model results saved at otus_results-dataset=cms_doubleelectron_mg5_dyee_test.npz
Training history saved at otus_history-dataset=cms_doubleelectron_mg5_dyee.npz
```

12 Validation Plots

[94]:
```
from func_utils import Zboson_mass

arrs = all_arrs['val']

m_x = Zboson_mass(arrs['x'])
m_rec = Zboson_mass(arrs['x_reconstructed'])
m_zdec = Zboson_mass(arrs['z_decoded'])

def summarize_mass(name, m):
    bins_summary = np.linspace(70, 110, 81)
    counts, edges = np.histogram(m, bins=bins_summary)
    peak = 0.5 * (edges[np.argmax(counts)] + edges[np.argmax(counts) + 1])
    print(
        f'{name:12s} '
        f'mean={np.mean(m):7.3f}, '
        f'std={np.std(m):7.3f}, '
        f'peak={peak:7.3f}, '
        f'frac_70_110={np.mean((m > 70) & (m < 110)):6.3f}'
    )

summarize_mass('CMS x', m_x)
summarize_mass('x-z-x', m_rec)
summarize_mass('z-x', m_zdec)

fig, ax = plt.subplots(figsize=(8, 5.5), constrained_layout=True)
bins = np.linspace(70, 110, 81)

ax.hist(m_x, bins=bins, histtype='step', linewidth=2, label='CMS x')
ax.hist(m_rec, bins=bins, histtype='step', linewidth=2, label='x -> z -> x')
ax.hist(m_zdec, bins=bins, histtype='step', linewidth=2, label='z -> x')
ax.axvline(91.1880, linestyle='--', linewidth=1.5, label=r'$m_Z = 91.1880$ GeV')

ax.set_xlabel(r'$m_{ee}$ [GeV]')
ax.set_ylabel('Events')
ax.set_xlim(70, 110)
ax.set_title(r'CMS DoubleElectron validation: $x$-space $Z\to ee$ mass')
ax.legend(frameon=False)
plt.show()
```

```
CMS x        mean= 90.593, std=  5.866, peak= 91.750, frac_70_110= 1.000
x-z-x        mean= 90.911, std=  6.268, peak= 91.750, frac_70_110= 0.994
z-x          mean= 93.253, std= 10.428, peak= 93.750, frac_70_110= 0.938
```

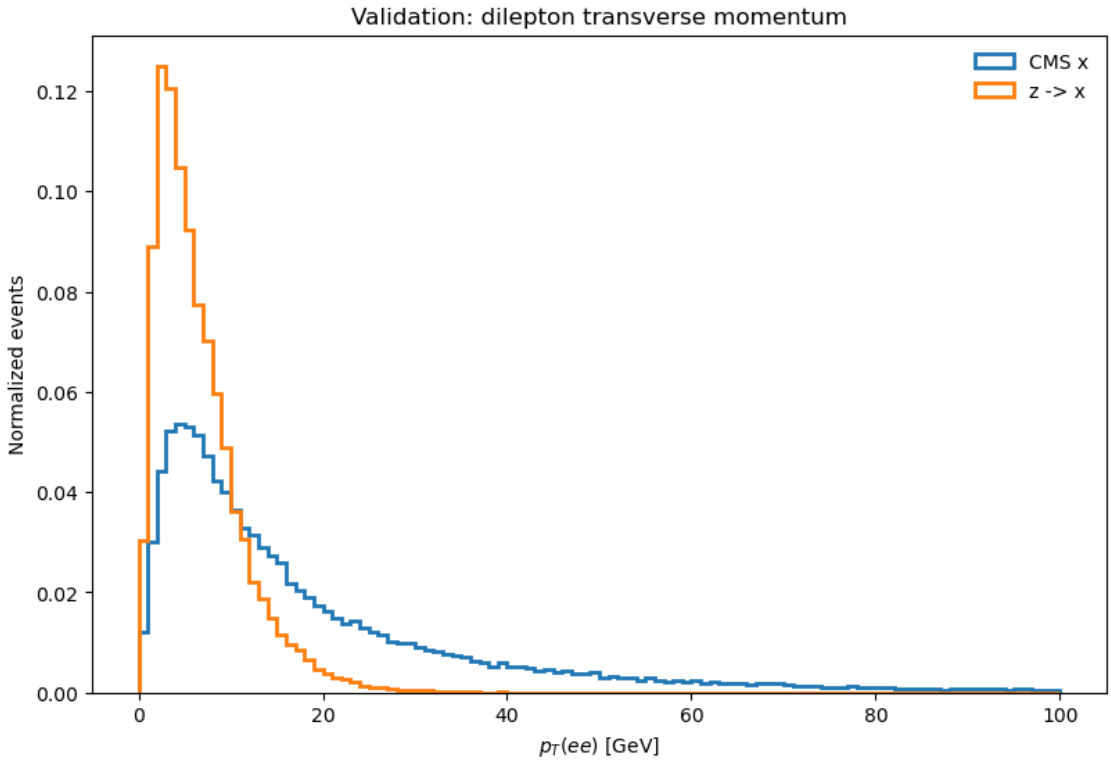


12.1 Optional generator diagnostic: dilepton transverse momentum

A pure LO MG5 `pp -> e+ e-` prior has very small dilepton recoil before OTUS. This check helps diagnose whether the generated `z -> x` events learn CMS-like transverse recoil.

[95]:
```
def dilepton_pt(y):
    px = y[:, 0] + y[:, 4]
    py = y[:, 1] + y[:, 5]
    return np.sqrt(px**2 + py**2)

x_ptll = dilepton_pt(arrs['x'])
zdec_ptll = dilepton_pt(arrs['z_decoded'])

fig, ax = plt.subplots(figsize=(8, 5.5), constrained_layout=True)
bins = np.linspace(0, 100, 101)

ax.hist(x_ptll, bins=bins, histtype='step', linewidth=2, density=True,␣
 ↪label='CMS x')
ax.hist(zdec_ptll, bins=bins, histtype='step', linewidth=2, density=True,␣
 ↪label='z -> x')

ax.set_xlabel(r'$p_T(ee)$ [GeV]')
```

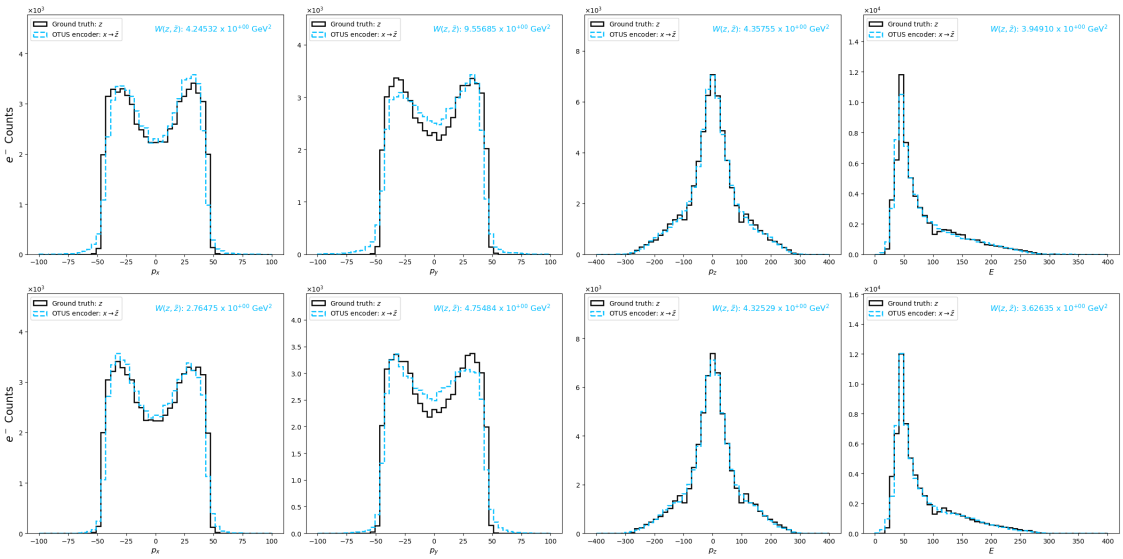
```
ax.set_ylabel('Normalized events')
ax.set_title(r'Validation: dilepton transverse momentum')
ax.legend(frameon=False)
plt.show()

print(f'CMS pT(ee) mean:  {np.mean(x_ptll):.4f} GeV')
print(f'z -> x pT(ee) mean: {np.mean(zdec_ptll):.4f} GeV')
```




```
CMS pT(ee) mean:  20.7032 GeV
z -> x pT(ee) mean: 6.4898 GeV
```

[96]:
```
from plot_utils import plotFunction

dataList = [arrs['z'], arrs['x_encoded']]
pltDim = (2, 4)
numBins = 50
binsList = [np.linspace(-100., 100., numBins),
            np.linspace(-100., 100., numBins),
            np.linspace(-400., 400., numBins),
            np.linspace(0., 400., numBins),
            np.linspace(-100., 100., numBins),
            np.linspace(-100., 100., numBins),
            np.linspace(-400., 400., numBins),
```

```
            np.linspace(0., 400., numBins)]
particleNameList = [r'$e^-$'] * 4 + [r'$e^+$'] * 4
nameList = [r'$p_x$', r'$p_y$', r'$p_z$', r'$E$', r'$p_x$', r'$p_y$', r'$p_z$',␣
 ↪r'$E$']

plotFunction(dataList=dataList, pltDim=pltDim, binsList=binsList,
             particleNameList=particleNameList, nameList=nameList)
```



[97]:
```
dataList = [arrs['x'], arrs['x_reconstructed'], arrs['z_decoded']]

plotFunction(dataList=dataList, pltDim=pltDim, binsList=binsList,
             particleNameList=particleNameList, nameList=nameList)
```